JSON REFINER ADVANCED EDITION

Code:

```
# -*- coding: utf-8 -*-
"""granite-3.3-2b-instruct.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/#fileId=https%3A//huggingface.co/ibm-granite/granite-3.3-2b-instruct.ipynb
"""

!pip install -U transformers

"""## Local Inference on GPU
Model page: https://huggingface.co/ibm-granite/granite-3.3-2b-instruct

⚠ If the generated code snippets do not work, please open an issue on either the [model repo](https://huggingface.co/ibm-granite/granite-3.3-2b-instruct)
                                and/or on [huggingface.js](https://github.com/huggingface/huggingface.js/blob/main/packages/tasks/src/model-libraries-snippets.ts) 🙏
"""

# Use a pipeline as a high-level helper
from transformers import pipeline
```

```
pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
pipe(messages)

# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
        messages,
        add_generation_prompt=True,
        tokenize=True,
        return_dict=True,
        return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

!pip install -q gradio json5 jsonschema python-dotenv pydantic

import gradio as gr
```

```
import json
import re
import ast
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Any, Tuple, Optional
import json5
from jsonschema import validate, ValidationError, Draft7Validator
from enum import Enum

# ==================================================================================
# JSON REFINER - ADVANCED VERSION WITH PREMIUM FEATURES (FIXED)
# ==================================================================================

class CaseStyle(Enum):
    """Case conversion styles"""
    SNAKE_CASE = "snake_case"
    CAMEL_CASE = "camelCase"
    KEBAB_CASE = "kebab-case"
    PASCAL_CASE = "PascalCase"

class JSONRefinerCore:
    """Advanced JSON parsing and refinement engine"""

    def __init__(self):
        self.conversion_log = []
```

```
    self.validation_errors = []
    self.transformation_history = []

@staticmethod
def to_snake_case(text: str) -> str:
    """Convert text to snake_case"""
    text = re.sub(r'[\s\-]+', '_', text)
    text = re.sub(r'([a-z])([A-Z])', r'\1_\2', text)
    text = re.sub(r'[^a-zA-Z0-9_]', '', text)
    return text.lower()

@staticmethod
def to_camel_case(text: str) -> str:
    """Convert text to camelCase"""
    snake = JSONRefinerCore.to_snake_case(text)
    parts = snake.split('_')
    return parts[0].lower() + ''.join(word.capitalize() for word in parts[1:])

@staticmethod
def to_kebab_case(text: str) -> str:
    """Convert text to kebab-case"""
    snake = JSONRefinerCore.to_snake_case(text)
    return snake.replace('_', '-')

@staticmethod
def to_pascal_case(text: str) -> str:
    """Convert text to PascalCase"""
    camel = JSONRefinerCore.to_camel_case(text)
    return camel[0].upper() + camel[1:] if camel else ""
```

```
@staticmethod
def normalize_key(key: str, case_style: CaseStyle = CaseStyle.SNAKE_CASE) -> str:
    """Normalize key name"""
    key = key.strip()

    if case_style == CaseStyle.SNAKE_CASE:
        return JSONRefinerCore.to_snake_case(key)
    elif case_style == CaseStyle.CAMEL_CASE:
        return JSONRefinerCore.to_camel_case(key)
    elif case_style == CaseStyle.KEBAB_CASE:
        return JSONRefinerCore.to_kebab_case(key)
    elif case_style == CaseStyle.PASCAL_CASE:
        return JSONRefinerCore.to_pascal_case(key)
    return key

@staticmethod
def infer_type(value: str) -> Any:
    """Infer and convert value to correct JSON type"""
    value = value.strip()

    if value.lower() in ['null', 'none', 'nil', 'undefined', 'n/a', 'na']:
        return None

    if value.lower() in ['true', 'yes', 'on', '1']:
        return True
    if value.lower() in ['false', 'no', 'off', '0']:
        return False
```

```
        try:
            if '.' not in value:
                return int(value)
        except ValueError:
            pass

        try:
            return float(value)
        except ValueError:
            pass

        if value.startswith('[') and value.endswith(']'):
            try:
                return json.loads(value)
            except:
                pass

        if value.startswith('{') and value.endswith('}'):
            try:
                return json.loads(value)
            except:
                pass

        return value

    @staticmethod
    def parse_key_value_text(text: str, case_style: CaseStyle = CaseStyle.SNAKE_CASE) -> Tuple[Dict, List[str]]:
        """Parse key-value text into dictionary"""
```

```
result = {}
errors = []
duplicates = []

lines = text.split('\n')

for line_num, line in enumerate(lines, 1):
    line = line.strip()

    if not line or line.startswith('#'):
        continue

    if ':' not in line:
        errors.append(f"Line {line_num}: No colon separator found - '{line}'")
        continue

    try:
        key, value = line.split(':', 1)
        key = key.strip()
        value = value.strip()

        if not key:
            errors.append(f"Line {line_num}: Empty key found")
            continue

        normalized_key = JSONRefinerCore.normalize_key(key, case_style)

        if normalized_key in result:
```

```
                duplicates.append(f"Line {line_num}: Duplicate key '{normalized_key}' (original: '{key}')")
                continue

            converted_value = JSONRefinerCore.infer_type(value)
            result[normalized_key] = converted_value

        except Exception as e:
            errors.append(f"Line {line_num}: Parse error - {str(e)}")

    all_errors = errors + duplicates
    return result, all_errors

@staticmethod
def validate_json_schema(data: Dict, schema: Dict) -> Tuple[bool, List[str]]:
    """Validate JSON against schema"""
    errors = []
    try:
        validate(instance=data, schema=schema)
        return True, []
    except ValidationError as e:
        return False, [str(e)]
    except Exception as e:
        return False, [f"Schema validation error: {str(e)}"]

@staticmethod
def check_required_fields(data: Dict, required_fields: List[str]) -> Tuple[bool, List[str]]:
    """Check for required fields"""
    errors = []
```

```
    missing = []
    for field in required_fields:
        if field not in data:
            missing.append(field)

    if missing:
        return False, [f"Missing required fields: {', '.join(missing)}"]
    return True, []

@staticmethod
def flatten_json(data: Dict, parent_key: str = '', sep: str = '.') -> Dict:
    """Flatten nested JSON"""
    items = []
    for k, v in data.items():
        new_key = f"{parent_key}{sep}{k}" if parent_key else k
        if isinstance(v, dict):
            items.extend(JSONRefinerCore.flatten_json(v, new_key, sep=sep).items())
        elif isinstance(v, list):
            items.append((new_key, v))
        else:
            items.append((new_key, v))
    return dict(items)

@staticmethod
def unflatten_json(data: Dict, sep: str = '.') -> Dict:
    """Unflatten dot-notation JSON"""
    result = {}
    for key, value in data.items():
        parts = key.split(sep)
```

```
        current = result
        for part in parts[:-1]:
            if part not in current:
                current[part] = {}
            current = current[part]
        current[parts[-1]] = value
    return result

@staticmethod
def merge_json_objects(*objects: Dict) -> Dict:
    """Deep merge multiple JSON objects"""
    result = {}
    for obj in objects:
        for key, value in obj.items():
            if key in result and isinstance(result[key], dict) and isinstance(value, dict):
                result[key] = JSONRefinerCore.merge_json_objects(result[key], value)
            else:
                result[key] = value
    return result

@staticmethod
def remove_null_values(data: Dict, recursive: bool = True) -> Dict:
    """Remove null/None values from JSON"""
    if not recursive:
        return {k: v for k, v in data.items() if v is not None}

    result = {}
    for k, v in data.items():
        if v is None:
```

```
            continue
        elif isinstance(v, dict):
            result[k] = JSONRefinerCore.remove_null_values(v, recursive=True)
        else:
            result[k] = v
    return result

@staticmethod
def get_json_stats(data: Dict) -> Dict[str, Any]:
    """Get statistics about JSON structure"""
    def count_items(obj):
        if isinstance(obj, dict):
            return len(obj), sum(count_items(v)[0] for v in obj.values())
        elif isinstance(obj, list):
            return len(obj), sum(count_items(item)[0] if isinstance(item, (dict, list)) else 0 for
item in obj)
        return 0, 0

    top_level, total = count_items(data)

    return {
        'top_level_keys': top_level,
        'total_keys': total,
        'has_nested_objects': any(isinstance(v, dict) for v in data.values()),
        'has_arrays': any(isinstance(v, list) for v in data.values()),
        'has_null_values': any(v is None for v in data.values()),
        'value_types': list(set(type(v).__name__ for v in data.values()))
    }
```

```
# Initialize core engine
refiner = JSONRefinerCore()
transformation_history = []

def refine_json(text_input: str, case_style: str = "snake_case", remove_nulls: bool = False,
                flatten: bool = False, pretty_print: bool = True) -> Tuple[str, str, str]:
    """Main refinement function"""
    global transformation_history

    if not text_input.strip():
        return "", "❌ Empty input provided!", ""

    try:
        case_enum = CaseStyle[case_style.upper().replace('-', '_')]
        parsed_data, parse_errors = refiner.parse_key_value_text(text_input, case_enum)

        if remove_nulls:
            parsed_data = refiner.remove_null_values(parsed_data)

        if flatten:
            parsed_data = refiner.flatten_json(parsed_data)

        if pretty_print:
            json_output = json.dumps(parsed_data, indent=2, ensure_ascii=False)
        else:
            json_output = json.dumps(parsed_data, ensure_ascii=False)

        stats = refiner.get_json_stats(parsed_data)
        stats_text = f"""
```

```
📊 **JSON Statistics:**
- Top-level Keys: {stats['top_level_keys']}
- Total Keys: {stats['total_keys']}
- Has Nested Objects: {stats['has_nested_objects']}
- Has Arrays: {stats['has_arrays']}
- Has Null Values: {stats['has_null_values']}
- Value Types: {', '.join(stats['value_types'])}
"""

        status_msg = "✅ Successfully converted to JSON!"
        if parse_errors:
            status_msg += f"\n\n⚠️ **Warnings ({len(parse_errors)}):**\n"
            status_msg += "\n".join(f"• {err}" for err in parse_errors[:5])
            if len(parse_errors) > 5:
                status_msg += f"\n• ... and {len(parse_errors) - 5} more errors"

        transformation_history.append({
            'timestamp': datetime.now().isoformat(),
            'input': text_input,
            'output': json_output,
            'errors': parse_errors,
            'stats': stats
        })

        return json_output, status_msg, stats_text

    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""
```

```
def validate_json(json_text: str, schema_text: str = "", required_fields: str = "") -> Tuple[str, str]:
    """Validate JSON against schema or required fields"""
    try:
        data = json.loads(json_text)

        validation_results = []

        if required_fields.strip():
            fields = [f.strip() for f in required_fields.split(',')]
            is_valid, errors = refiner.check_required_fields(data, fields)
            if not is_valid:
                validation_results.extend(errors)
            else:
                validation_results.append("✅ All required fields present")

        if schema_text.strip():
            schema = json.loads(schema_text)
            is_valid, errors = refiner.validate_json_schema(data, schema)
            if not is_valid:
                validation_results.extend(errors)
            else:
                validation_results.append("✅ Schema validation passed")

        if not validation_results:
            validation_results.append("✅ Valid JSON structure")

        result_text = "\n".join(f"• {res}" for res in validation_results)
        return result_text, "✅ Validation complete!"
```

```
    except json.JSONDecodeError as e:
        return f"❌ Invalid JSON: {str(e)}", "❌ JSON parse error"
    except Exception as e:
        return f"❌ Error: {str(e)}", "❌ Validation error"

def convert_case(json_text: str, target_case: str) -> Tuple[str, str]:
    """Convert JSON keys to different case styles"""
    try:
        data = json.loads(json_text)
        case_enum = CaseStyle[target_case.upper().replace('-', '_')]

        def convert_keys(obj):
            if isinstance(obj, dict):
                return {
                    refiner.normalize_key(k, case_enum): convert_keys(v)
                    for k, v in obj.items()
                }
            elif isinstance(obj, list):
                return [convert_keys(item) for item in obj]
            return obj

        converted = convert_keys(data)
        output = json.dumps(converted, indent=2, ensure_ascii=False)
        return output, f"✅ Converted to {target_case}!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"
```

```
def merge_multiple_json(json_text1: str, json_text2: str, json_text3: str = "") -> Tuple[str,
str]:
    """Merge multiple JSON objects"""
    try:
        objects = []
        for json_text in [json_text1, json_text2, json_text3]:
            if json_text.strip():
                objects.append(json.loads(json_text))

        if not objects:
            return "", "❌ No JSON objects to merge"

        merged = refiner.merge_json_objects(*objects)
        output = json.dumps(merged, indent=2, ensure_ascii=False)
        return output, f"✅ Merged {len(objects)} JSON objects!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"

def flatten_json_func(json_text: str) -> Tuple[str, str]:
    """Flatten JSON structure"""
    try:
        data = json.loads(json_text)
        flattened = refiner.flatten_json(data)
        output = json.dumps(flattened, indent=2, ensure_ascii=False)
        return output, "✅ JSON flattened!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"

def unflatten_json_func(json_text: str) -> Tuple[str, str]:
```

```
    """Unflatten JSON structure"""
    try:
        data = json.loads(json_text)
        unflattened = refiner.unflatten_json(data)
        output = json.dumps(unflattened, indent=2, ensure_ascii=False)
        return output, "✅ JSON unflattened!"
    except Exception as e:
        return "", f"❌ Error: {str(e)}"

def clear_history():
    """Clear transformation history"""
    global transformation_history
    transformation_history = []
    return "✅ History cleared!"

def download_history() -> str:
    """Download transformation history"""
    if not transformation_history:
        return "❌ No history to download"

    report = "JSON REFINER - TRANSFORMATION HISTORY\n"
    report += f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
    report += "="*80 + "\n\n"

    for i, item in enumerate(transformation_history, 1):
        report += f"Transformation #{i}\n"
        report += f"Timestamp: {item['timestamp']}\n"
        report += f"Input:\n{item['input']}\n\n"
        report += f"Output:\n{item['output']}\n\n"
```

```
        if item['errors']:
            report += f"Errors: {', '.join(item['errors'])}\n\n"
        report += "-"*80 + "\n\n"

    return report

print("🚀 Loading JSON Refiner Advanced...")

with gr.Blocks(
    title="JSON Refiner - Advanced Edition"
) as demo:
    demo.load(None, js="""
        () => {
            document.body.style.background = 'linear-gradient(135deg, #0f172a 0%, #1e293b
100%)';
            const style = document.createElement('style');
            style.textContent = `
                :root {
                    --primary-color: #667eea;
                    --secondary-color: #f5576c;
                }
                body {
                    background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%) !important;
                    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
                }
                .gradio-container {
                    background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%) !important;
                    max-width: 1400px !important;
                }
```

```
            h1, h2, h3 { color: #f1f5f9 !important; }
            .code-block { background: #1e293b !important; border: 2px solid #667eea !important; }
        `;
        document.head.appendChild(style);
    }
""")

gr.Markdown("""
# 🎯 JSON Refiner Advanced Edition
## Professional JSON Processing Tool

**Features:** Type Inference • Schema Validation • Case Conversion • Flattening • Merging • History Tracking
""")

with gr.Tabs():

    # ============= TAB 1: Core Refining =============
    with gr.TabItem("🔧 Core Refining"):
        with gr.Row():
            with gr.Column(scale=1):
                gr.Markdown("### 📝 Input Format")
                text_input = gr.Textbox(
                    label="Key-Value Text",
                    lines=12,
                    value="name: John Doe\nage: 28\nis_active: true\nbalance: 1500.50\nemail: john@example.com\ntags: [\"admin\", \"user\"]\nmetadata: {\"level\": 5}"
                )
```

```
        case_style = gr.Radio(
            ["snake_case", "camelCase", "kebab-case", "PascalCase"],
            value="snake_case",
            label="🎨 Key Case Style"
        )

        with gr.Row():
            remove_nulls = gr.Checkbox(label="Remove Nulls", value=False)
            flatten = gr.Checkbox(label="Flatten", value=False)
            pretty_print = gr.Checkbox(label="Pretty Print", value=True)

        refine_btn = gr.Button("✨ Refine JSON", size="lg", variant="primary")

    with gr.Column(scale=1):
        gr.Markdown("### 📊 Output")
        json_output = gr.Code(
            language="json",
            label="Refined JSON",
            lines=12
        )

        status_text = gr.Markdown(label="Status")
        stats_text = gr.Markdown(label="Statistics")

refine_btn.click(
    refine_json,
    inputs=[text_input, case_style, remove_nulls, flatten, pretty_print],
    outputs=[json_output, status_text, stats_text]
)
```

```
# ============= TAB 2: Validation =============
with gr.TabItem("✅ Validation"):
    gr.Markdown("### 🔍 Validate JSON Structure")

    with gr.Row():
        with gr.Column(scale=1):
            json_to_validate = gr.Code(
                language="json",
                label="JSON to Validate",
                lines=10,
                value='{"name": "John", "age": 28}'
            )

            required_fields_input = gr.Textbox(
                label="Required Fields (comma-separated)",
                value="name, age, email"
            )

        with gr.Column(scale=1):
            schema_input = gr.Code(
                language="json",
                label="JSON Schema (optional)",
                lines=10,
                value='{"type": "object", "properties": {"name": {"type": "string"}}}'
            )

            validate_btn = gr.Button("🔍 Validate", size="lg", variant="primary")
```

```
    validation_results = gr.Markdown(label="Validation Results")
    validation_status = gr.Markdown()

    validate_btn.click(
        validate_json,
        inputs=[json_to_validate, schema_input, required_fields_input],
        outputs=[validation_results, validation_status]
    )

# ============= TAB 3: Case Conversion =============
with gr.TabItem("🎨 Case Conversion"):
    gr.Markdown("### 🔄 Convert JSON Key Cases")

    with gr.Row():
        with gr.Column(scale=1):
            json_for_case = gr.Code(
                language="json",
                label="Input JSON",
                lines=10,
                value='{"firstName": "John", "lastName": "Doe"}'
            )

            target_case = gr.Radio(
                ["snake_case", "camelCase", "kebab-case", "PascalCase"],
                value="camelCase",
                label="Target Case Style"
            )

            case_convert_btn = gr.Button("🔄 Convert", size="lg", variant="primary")
```

```
        with gr.Column(scale=1):
            case_output = gr.Code(
                language="json",
                label="Converted JSON",
                lines=10
            )
            case_status = gr.Markdown()

    case_convert_btn.click(
        convert_case,
        inputs=[json_for_case, target_case],
        outputs=[case_output, case_status]
    )

# ============= TAB 4: Merging =============
with gr.TabItem("🔗 Merge JSON"):
    gr.Markdown("### 🔀 Merge Multiple JSON Objects")

    with gr.Row():
        with gr.Column(scale=1):
            json1 = gr.Code(
                language="json",
                label="JSON #1",
                lines=8,
                value='{"name": "John"}'
            )
            json2 = gr.Code(
                language="json",
```

```
            label="JSON #2",
            lines=8,
            value='{"age": 28}'
        )
        json3 = gr.Code(
            language="json",
            label="JSON #3 (optional)",
            lines=8,
            value='{"email": "john@example.com"}'
        )

        merge_btn = gr.Button("🔗 Merge", size="lg", variant="primary")

    with gr.Column(scale=1):
        merge_output = gr.Code(
            language="json",
            label="Merged JSON",
            lines=10
        )
        merge_status = gr.Markdown()

merge_btn.click(
    merge_multiple_json,
    inputs=[json1, json2, json3],
    outputs=[merge_output, merge_status]
)

# ============= TAB 5: Flattening =============
with gr.TabItem("📋 Flatten/Unflatten"):
```

```
with gr.Tabs():
    with gr.TabItem("Flatten"):
        gr.Markdown("### 📄 Flatten Nested JSON")

        with gr.Row():
            flatten_input = gr.Code(
                language="json",
                label="Nested JSON",
                lines=10,
                value='{"user": {"name": "John", "age": 28}}'
            )
            flatten_btn = gr.Button("Flatten", size="lg", variant="primary")

        flatten_out = gr.Code(
            language="json",
            label="Flattened JSON",
            lines=10
        )
        flatten_status = gr.Markdown()

        flatten_btn.click(
            flatten_json_func,
            inputs=[flatten_input],
            outputs=[flatten_out, flatten_status]
        )

    with gr.TabItem("Unflatten"):
        gr.Markdown("### 📄 Unflatten Dot-Notation JSON")
```

```
        with gr.Row():
            unflatten_input = gr.Code(
                language="json",
                label="Flattened JSON",
                lines=10,
                value='{"user.name": "John", "user.age": 28}'
            )
            unflatten_btn = gr.Button("Unflatten", size="lg", variant="primary")

        unflatten_out = gr.Code(
            language="json",
            label="Nested JSON",
            lines=10
        )
        unflatten_status = gr.Markdown()

        unflatten_btn.click(
            unflatten_json_func,
            inputs=[unflatten_input],
            outputs=[unflatten_out, unflatten_status]
        )

# ============= TAB 6: History =============
with gr.TabItem("📜 History"):
    gr.Markdown("### 📊 Transformation History")

    with gr.Row():
        history_btn = gr.Button("📥 Download History", size="lg", variant="secondary")
        clear_history_btn = gr.Button("🗑️ Clear History", size="lg", variant="stop")
```

```
    history_output = gr.Textbox(
        label="History Report",
        lines=15
    )

    history_status = gr.Markdown()

    history_btn.click(
        download_history,
        outputs=[history_output]
    )

    clear_history_btn.click(
        clear_history,
        outputs=[history_status]
    )

# ============= TAB 7: Guide =============
with gr.TabItem("📚 Guide"):
    gr.Markdown("""
    # 📖 JSON Refiner - Complete Guide

    ## Input Format
    Use simple **Key: Value** format:
    ```
    name: John Doe
    age: 28
    is_active: true
```
```

balance: 1500.50
```

## Type Inference

**Automatic type detection:**

- `true`, `false`, `yes`, `no` → Boolean
- `null`, `none`, `undefined` → Null
- `123`, `456` → Integer
- `3.14`, `2.5` → Float
- `[1, 2, 3]` → Array (if valid JSON)
- `{"key": "value"}` → Object (if valid JSON)
- Everything else → String

## Case Styles

- **snake_case**: `user_name`, `first_name`
- **camelCase**: `userName`, `firstName`
- **kebab-case**: `user-name`, `first-name`
- **PascalCase**: `UserName`, `FirstName`

## Features

### ✅ Validation

- Check required fields
- Validate against JSON Schema
- Detect structural issues

### 🎨 Case Conversion

- Convert between case styles
- Recursive conversion for nested objects
```

📎 Merging

- Deep merge multiple JSON objects
- Conflict resolution (last one wins)

📋 Flattening

- Flatten nested objects to dot-notation
- Unflatten back to nested structure

📜 History

- Track all transformations
- Download transformation history

Examples

Basic Conversion

Input:

```
name: John
age: 30
active: true
```

Output (snake_case):

```json
{
  "name": "John",
  "age": 30,
  "active": true
```

}
```
```